2D & Multidimensional Arrays

Arrays

Definition: -

A multidimensional array - an array of arrays.

General form of declaring N-dimensional arrays -

datatype arr_name[size1][size2]....[sizeN];

datatype: Type of data to be stored in the array.

arr_name: Name assigned to the array.

size1, size2,…, sizeN: Size of each dimension.

Eg: -

Two-dimensional array: *int two_d[10][20];*

Three-dimensional array: *int three_d[10][20][30];*

Arrays

Two-Dimensional Arrays

- A 2D array is also known as a matrix (a table of rows and columns).

Syntax: datatype arrayname[dimension1][dimension2];

Eg:-

A 2D array with **m** rows and **n** columns can be created as:

datatype arr_name[m][n]; where m=no. of rows, n = no. of columns

int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

The first dimension represents the number of rows [2], while the second dimension represents the number of columns [3]. The values are placed in row-order, and can be visualized like this:

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

Initialization of 2D arrays

int arr[3][4] = {0, 1 ,2 ,3 ,4 , 5 , 6 , 7 , 8 , 9 , 10 , 11}

Or

int arr[3][4] = {{0, 1, 2, 3}, {4, 5, 6, 7}, {8, 9, 10, 11}};

Note: *The number of elements in initializer list should always be either less than or equal to the total number of elements in the array.*

- The elements will be stored in the array from left to right and top to bottom.
- So, the first 4 elements from the left will be filled in the first row, the next 4 elements in the second row, and so on.
- This is clearly shown in the second syntax where each set of inner braces represents one row.
- In list initialization, we can skip specifying the size of the row. The compiler will automatically deduce it in this case. So, the below declaration is valid.
- type arr_name[][n] = {…values…};
- It is still compulsory to define the number of columns.

Access the Elements of a 2D Array

- To access an element of a two-dimensional array, you must specify the index number of both the row and column.
- This statement accesses the value of the element in the **first row (0)** and **third column (2)** of the **matrix** array.

Example

```
int matrix[2][3] = { {10, 20, 30}, {40, 50, 60} };

printf("%d", matrix[0][1]);  // Outputs 20
```

Updation

```
matrix[0][0] = 15;
```

Loop Through a 2D Array

- To loop through a multi-dimensional array, you need one loop for each of the array's dimensions.

For 2D array

```
int matrix[2][3] = { {10, 20, 30}, {40, 50, 60} };

int i, j;
for (i = 0; i < 2; i++) {
  for (j = 0; j < 3; j++) {
    printf("%d\n", matrix[i][j]);
  }
}
```

Loop Through a 2D Array

```
#include <stdio.h>
int main()
{
int arr[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } }; /* Create and initialize an array with 3 rows and 2 columns */

for (int i = 0; i < 3; i++) /*Print each array element's value */
{
    for (int j = 0; j < 2; j++)
     {
        printf("arr[%d][%d]: %d ", i, j, arr[i][j]);
     }
     printf("\n");
}
return 0;
}
```

Sum of two matrices of order 2*2

```
#include <stdio.h>
int main()
{ int a[2][2], b[2][2], result[2][2];
// Taking input1 using nested for loop
printf("Enter elements of 1st matrix\n");
for (int i = 0; i < 2; ++i)
    for (int j = 0; j < 2; ++j)
    {
      printf("Enter a%d%d: ", i + 1, j + 1);
      scanf("%d", &a[i][j]);
    }
// Taking input2 using nested for loop
  printf("Enter elements of 2nd matrix\n");
    for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
        {
          printf("Enter b%d%d: ", i + 1, j + 1);
          scanf("%d", &b[i][j]);
        }
// adding corresponding elements of two arrays
for (int i = 0; i < 2; ++i)
    for (int j = 0; j < 2; ++j)
    {
      result[i][j] = a[i][j] + b[i][j];
    }
  // Displaying the sum
  printf("\nSum Of Matrix:");
  for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
        {
           printf("%d\t", result[i][j]);
        }
           printf("\n");
return 0;
}
```

Multiplication of two matrices

```
#include<stdio.h>
int main()
{
int mat1[3][3] = { {2, 4, 1} , {2, 3, 9} , {3, 1, 8} };
int mat2[3][3] = { {1, 2, 3} , {3, 6, 1} , {2, 4, 7} };
int mat3[3][3], sum=0, i, j, k;
for(i=0; i<3; i++)
{
  for(j=0; j<3; j++)
     {
        sum=0;
        for(k=0; k<3; k++)
        sum = sum + mat1[i][k] * mat2[k][j]; mat3[i][j] = sum;
     }
}
printf("\nMatrix 1 ...\n");
for(i=0; i<3; i++)
{
   for(j=0; j<3; j++)
        printf("%d\t", mat1[i][j]);
    printf("\n");
}
printf("\nMatrix 2 ...\n");
for(i=0; i<3; i++)
{
   for(j=0; j<3; j++)
         printf("%d\t", mat2[i][j]);
    printf("\n");
}
printf("\nMultiplication of the two given Matrices: \n");
for(i=0; i<3; i++)
 {
     for(j=0; j<3; j++)
          printf("%d\t", mat3[i][j]);
     printf("\n");
  }
   return 0;
}
```